DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 3 – CO3

Student Details	Information
Student A	Mohammed Affan M
Register Number	192572169
Student B	_____
Student B Register Number	_____
Mode	Pair Work – 2 Students per Group

Note: Both partners must understand every question. Student A/B labels indicate primary responsibilities, not exclusive knowledge. Add genuine experimental timings from your own computer; no fabricated measurements are included.

Q1 – Decomposition and Elimination in Ordered Data

Part A – Finding Maximum and Minimum

Divide the array into two halves. Each recursive call returns (minimum, maximum). For one element, return (x,x). For two elements, use one comparison. Combine two results with min(minL,minR) and max(maxL,maxR).

```
A[low...high]
          |
     divide at mid
     /          \\
left half     right half
(minL,maxL)   (minR,maxR)
     \          /
   combine min and max
          |
   (minimum, maximum)
```

Algorithm:

```
MINMAX(A, low, high):
    if low == high: return (A[low], A[low])
    if high == low+1:
        if A[low] < A[high]: return (A[low], A[high])
        else: return (A[high], A[low])
    mid = (low+high)//2
    (mn1,mx1) = MINMAX(A,low,mid)
    (mn2,mx2) = MINMAX(A,mid+1,high)
    return (min(mn1,mn2), max(mx1,mx2))
```

Comparison analysis: Finding min and max independently needs (n–1)+(n–1)=2n–2 comparisons. Pairwise/tournament min-max needs about 3n/2 comparisons; for even n, the standard pairwise method uses 3n/2–2 comparisons. This saves approximately n/2–1 comparisons. For an arbitrary recursive split, count comparisons according to its base cases; do not assume the exact even-n formula for every n.

Student A: Draw the split tree, explain that each node returns a min/max pair, show the combine operation, and derive the comparison count. **Student B:** Test random, sorted, reverse and duplicate arrays at sizes 10, 100, 1,000 and 10,000. Increment a counter for each element comparison and compare observed counts with the prediction.

Part B – Binary Search

The array must be sorted. Compare target with the middle element. If equal, return the index. If target is larger, discard the left half including mid; if smaller, discard the right half including mid. The discarded part cannot contain the target because the array is ordered.

```
BINARY-SEARCH(A, target):
    low=0; high=len(A)-1
    while low <= high:
        mid=(low+high)//2
        if A[mid] == target: return mid
        if A[mid] < target: low=mid+1
        else: high=mid-1
    return -1
```

Termination occurs when the target is found or low > high (empty search interval). Each step halves the remaining elements, giving O(log n) time and O(1) extra space. Student A should test successful middle/first/last targets and unsuccessful smaller/larger/missing targets on increasing sizes. Student B should explain the elimination proof and logarithmic growth.

Q2 – The Quality of Decomposition in Sorting

Merge Sort

Split the array into two nearly equal halves until single-element arrays remain. Merge the sorted halves by repeatedly taking the smaller front element. The recursion tree is balanced, independent of input order.

```
MERGE-SORT(A):
    if len(A)<=1: return A
    split A into L and R
```

```
    return MERGE(MERGE-SORT(L), MERGE-SORT(R))
```

Recurrence: T(n)=2T(n/2)+Θ(n), hence Θ(n log n). Merge Sort uses Θ(n) auxiliary memory and can be stable.

Quick Sort – selected pivot strategy: last element (Lomuto)

```
QUICK-SORT(A, low, high):
    if low < high:
        q = PARTITION(A, low, high)  # last element pivot
        QUICK-SORT(A, low, q-1)
        QUICK-SORT(A, q+1, high)
```

Balanced partitions are close to n/2 and n/2; recursion depth is Θ(log n), and time is Θ(n log n). Severely unbalanced partitions are 0 and n–1; recursion depth is Θ(n), and time becomes Θ(n²). With a last-element pivot, already sorted and reverse-sorted arrays can trigger this worst case.

Feature	Merge Sort	Quick Sort
Decomposition	Equal halves by index	Depends on pivot rank
Recursion height	Θ(log n)	Balanced Θ(log n), worst Θ(n)
Time	Θ(n log n)	Average Θ(n log n), worst Θ(n²)
Extra space	Θ(n) merge storage	Partition in place; recursion stack
Predictability	Predictable	Pivot-sensitive

Student A: Use shuffled/median-like pivot inputs for balanced partitions and sorted/reverse inputs for last-pivot imbalance. Record partition sizes and maximum recursion depth. **Student B:** Explain that Merge Sort's split is index-based and predictable, while Quick Sort depends on pivot quality; repeated bad partitions form a chain and cause quadratic work.

Joint design: Before choosing, examine input size and order, duplicate frequency, stability requirement, memory limits, worst-case latency, and whether randomised or median-based pivoting is acceptable. Prefer Merge Sort for predictable worst-case time; use Quick Sort when in-place practical performance matters and pivot strategy is robust.

Q3 – Karatsuba Multiplication

Split n-digit integers at m digits: x=a·10^m+b and y=c·10^m+d. Ordinary multiplication needs ac, ad, bc and bd. Karatsuba uses only three recursive products: z2=ac, z0=bd, z1=(a+b)(c+d)–z2–z0. Then xy=z2·10^(2m)+z1·10^m+z0.

```
KARATSUBA(x,y):
    if x or y is small: return x*y
    split x into a,b and y into c,d at m digits
    z2 = KARATSUBA(a,c)
    z0 = KARATSUBA(b,d)
    z1 = KARATSUBA(a+b,c+d)-z2-z0
    return z2*10^(2m)+z1*10^m+z0
```

Recurrence: T(n)=3T(n/2)+Θ(n). By the Master Theorem, T(n)=Θ(n^(log■3))≈Θ(n^1.585), compared with Θ(n²) for standard digit multiplication.

Student A: Explain the split, three products, cross-term reconstruction, recurrence and complexity. **Student B:** Test equal operands of 16, 32, 64, 128, 256, 512 and 1,024 digits; verify results, record multiplication counts and median time over repeated runs. Crossover is the size range where Karatsuba consistently becomes faster.

Karatsuba may lose for small inputs because recursion, splitting, temporary integers, additions and memory allocation have overhead. Implementations use a threshold and switch to ordinary multiplication below it. Better asymptotic complexity does not guarantee better performance at every size because constant factors and overhead matter.

Q4 – Strassen's Matrix Multiplication

For A and B divided into four blocks, conventional multiplication uses eight block products:

```
C11=A11B11+A12B21       C12=A11B12+A12B22
C21=A21B11+A22B21       C22=A21B12+A22B22
```

Strassen's seven products:

```
M1=(A11+A22)(B11+B22)   M2=(A21+A22)B11
M3=A11(B12-B22)         M4=A22(B21-B11)
M5=(A11+A12)B22         M6=(A21-A11)(B11+B12)
M7=(A12-A22)(B21+B22)

C11=M1+M4-M5+M7         C12=M3+M5
C21=M2+M4               C22=M1-M2+M3+M6
```

Expanding the seven expressions and cancelling terms reproduces the four conventional block equations, so the product is correct. Strassen recurrence is T(n)=7T(n/2)+Θ(n²), giving Θ(n^(log■7))≈Θ(n^2.807), compared with Θ(n³) conventional multiplication.

Padding: Pad a non-power-of-two square matrix with zero rows/columns to the next power of two, multiply, then crop the result. Padding can add work.

Student B: Explain the eight conventional products, seven Strassen products, reconstruction and zero padding. **Student A:** Test sizes 32, 64, 128, 256, 512 (as hardware allows), count scalar multiplications, repeat timings and discuss additions, recursion, padding, memory and cache.

Joint decision: Conventional multiplication is preferable for small matrices, irregular dimensions, memory constraints, simplicity or when Strassen overhead exceeds its benefit. Strassen is useful for sufficiently large matrices when experiments show a practical gain.

Q5 – Closest Pair Using Divide and Conquer

Brute force examines every unordered pair: $n(n-1)/2$ distance calculations, $\Theta(n^2)$. D&C; sorts points by x and y, divides at the median x-coordinate, and recursively returns the closest pair and distance from each half.

Let δ be the smaller recursive distance. Build a strip of points within δ of the dividing line and examine it in y-order. A closest cross-boundary pair may have one point in each half, so the strip must be checked.

```
CLOSEST-PAIR(P):
    sort points by x and by y
    if size <= 3: check all pairs
    divide into left and right halves
    solve each half recursively
    delta = smaller returned distance
    strip = points within delta of dividing line, sorted by y
    compare each strip point with only a constant number
        of following points in y-order
    return the best pair and distance
```

The strip does not require all-pairs comparison. Since each half already has no pair closer than δ, a geometric packing argument limits how many points can lie in the relevant 2δ-by-δ neighbourhood. In the standard 2D algorithm, each strip point needs comparison with at most the next seven points. Overall complexity is $\Theta(n \log n)$, assuming the y-order is maintained efficiently.

Student A: Explain Px/Py organisation, division, recursive return values, strip combine and boundary handling. **Student B:** Run brute force and D&C; on identical datasets of 100, 500, 1,000, 5,000 and 10,000 points; count distance calculations and record repeated runtimes. D&C; has reduced work when it returns the same closest distance with substantially fewer distance evaluations and lower growth.

Q6 – Advanced Divide-and-Conquer Mini-Investigations

A. Median of Medians – Student A

To find the k-th smallest item (k is 1-based), divide the array into groups of five, sort each group, and take each group's median. Recursively select the median of these medians as pivot. Partition into less-than, equal-to and greater-than groups; recurse only into the group containing rank k.

```
SELECT-MOM(A,k):
    if len(A)<=5: return sorted(A)[k-1]
    divide A into groups of five
    medians = median of each sorted group
    pivot = SELECT-MOM(medians, ceil(len(medians)/2))
    L = values < pivot; E = values == pivot; G = values > pivot
    if k <= len(L): return SELECT-MOM(L,k)
    if k <= len(L)+len(E): return pivot
    return SELECT-MOM(G,k-len(L)-len(E))
```

The pivot guarantees that a constant fraction (approximately 3n/10) of elements lie on either side, apart from small groups. Recurrence $T(n) \leq T(n/5)+T(7n/10)+O(n)=O(n)$. QuickSelect is expected $\Theta(n)$ with random pivots but worst-case $\Theta(n^2)$; deterministic first-element pivot on sorted data can be disastrous. Randomisation reduces the likelihood of repeated poor splits but cannot eliminate the theoretical worst case.

Student B verification: Pivot quality controls partition balance and how much input can be discarded. A central pivot makes selection fast; an extreme pivot leaves almost the entire array for the next call.

B. Meet-in-the-Middle Subset Sum – Student B

Split n values into halves L and R. Generate all subset sums for each half. Store right-half sums in a hash set. For each left sum x, check whether target–x exists in the right set.

```
MEET-IN-MIDDLE(A,target):
    split A into L and R
    left_sums = all subset sums of L
    right_sums = hash set of all subset sums of R
    for x in left_sums:
        if target-x in right_sums: return True
    return False
```

Each half has about 2^(n/2) subsets. Expected time and memory are O(2^(n/2)), compared with O(2^n) straightforward enumeration. The method uses extra memory to reduce search time: this is the time-memory trade-off.

Student A verification: The extra storage holds partial sums, allowing the algorithm to match two halves rather than enumerate every full subset.

Experimental Method and Results Tables to Complete

Use the same input for both algorithms in each comparison. Verify correctness first. Run at least five trials and report median time. Record computer model, Python version, input generation and operation-counter definition.

Experiment	Input sizes / conditions	Record
Min-Max	10, 100, 1,000, 10,000; random/sorted/reverse	Key comparisons
Binary Search	16, 32, 64, 128, 256, 512; hit/miss	Comparisons per query
Quick Sort	balanced-like; sorted; reverse; random	Partition sizes; max recursion depth
Karatsuba	16–1,024 digit operands	Multiplication count; median time
Strassen	32–512 square matrices	Scalar products; median time
Closest Pair	100–10,000 same point sets	Distance calculations; median time
Selection / Subset Sum	Several n and k/targets	Correctness; time; memory

Sample theoretical comparison: For closest-pair brute force, n=1,000 requires 499,500 unordered-pair distance checks; n=5,000 requires 12,497,500; n=10,000 requires 49,995,000. D&C; should grow approximately as n log n, but its exact count depends on data and implementation.

Conclusion

Divide and Conquer solves a problem by decomposing it into smaller subproblems and combining their results. Balanced decomposition gives predictable logarithmic recursion depth. Poor pivots can create quadratic behaviour. Karatsuba and Strassen reduce recursive multiplication counts, while closest pair and Meet-in-the-Middle reduce search work through structured decomposition. Practical decisions must consider overhead, memory, input characteristics and measured performance.

Pair Contribution Record

Question	Student A primary role	Student B primary role
Q1	Min-Max design; binary-search experiments	Min-Max validation; binary-search elimination
Q2	Quick Sort investigation	Merge Sort analysis
Q3	Karatsuba derivation	Crossover experiments
Q4	Strassen experiments	Algebra and correctness
Q5	Closest-pair decomposition	Comparative evidence
Q6	Median of Medians	Meet-in-the-Middle

Before submission: Fill in Student B details, add actual experimental measurements and plots, run correctness tests, and ensure both partners can explain all questions.